

Operator

日本工学院専門学校
デザインカレッジ



Operator（オペレータ）はメニュー内などに出てくる、個々の機能のこと。

左の図のようにメニューに配置することもできるし、スクリプトから関数のように直接呼び出して実行することもできる。

Operator (再び)

今回の目標

日本工学院専門学校
デザインカレッジ

- オペレータを自作する

TL1

01_05.Operator

TL1

01_05.Operator

コラム

日本工学院専門学校
デザインカレッジ

クラス名が

「TOPBAR_MT_my_menu」、「MYADDON_OT_stretch_vertex」
のような名前なのは、Blender 2.8 Python APIからの
命名規則に合わせている。

XXX_MT_XXX：メニュー

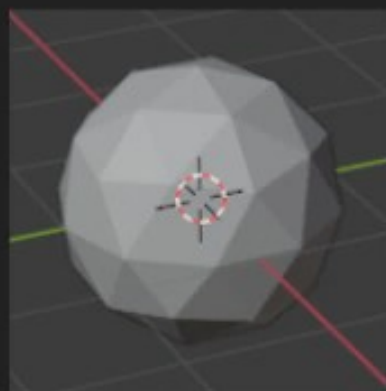
XXX_OT_XXX：オペレータ

クラスID(bl_idname)にも規則があり、メニューとオペレータで規則が異なる。
規則に合わせないと有効化時にWARNINGが表示される。

[参考サイト](#)

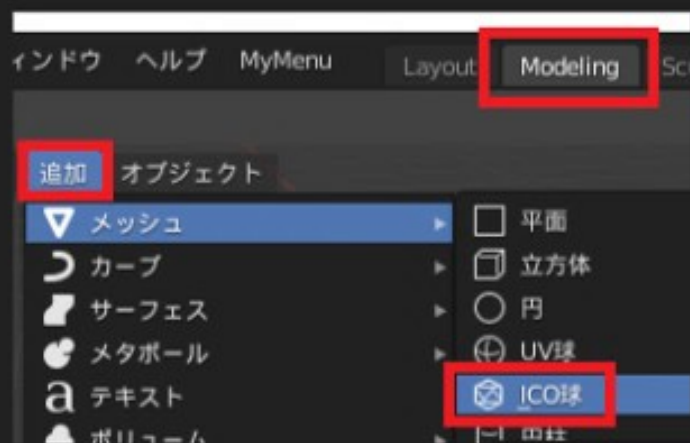
実行結果

日本工学院専門学校
デザインカレッジ



新たなメニュー項目「ICO球生成」を選択すると、ICO球オブジェクトが追加されるようになる。

ちなみにICO球を追加する機能は元々メニューにもあって、「Modeling」タブ→「追加」→「ICO球」でも出来る。



実行結果

日本工学院専門学校
デザインカレッジ



MyMenuにさらに項目が増えた。

別のオペレータクラス

日本工学院専門学校
デザインカレッジ

クラスを定義したら、さらに

- メニューへの登録
- Blenderに登録するクラスリストへの登録

を行う。

反復練習なのでコードは省略する。

今回の資料でやったことを参考に、自分で書いてみよう。

別のオペレータクラス

日本工学院専門学校
デザインカレッジ

```
level_editor.py
#オペレータ ICO球生成
class MYADDON_OT_create_ico_sphere(bpy.types.Operator):
    bl_idname = "myaddon.myaddon_ot_create_object"
    bl_label = "ICO球生成"
    bl_description = "ICO球を生成します"
    bl_options = {'REGISTER', 'UNDO'}

    # メニューを実行したときに呼ばれる関数
    def execute(self, context):
        bpy.ops.mesh.primitive_ico_sphere_add()
        print("ICO球を生成しました。")

        return {'FINISHED'}
```

練習としてさらに別のオペレータを作ってみる。

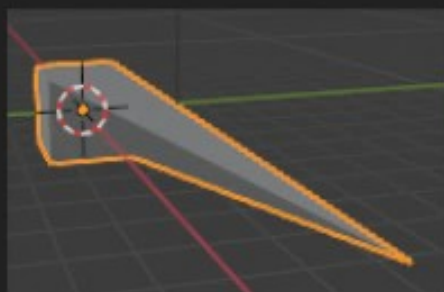
OPSモジュールのリファレンス

今回はメッシュ関連コマンドの中から、ICO球生成を呼び出すオペレータを作る。

反復練習

実行結果

日本工学院専門学校
デザインカレッジ



びろーん

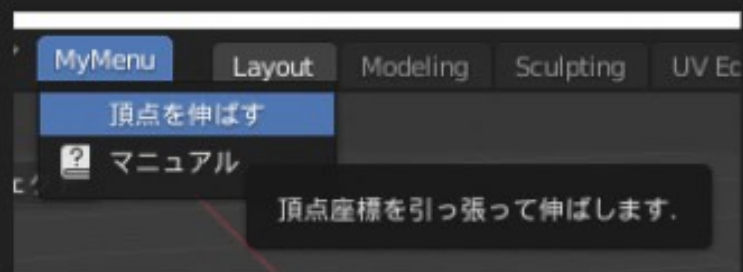
「頂点を伸ばす」を選択することにより、頂点が伸びていく。

また、Ctrl+Zで アンドゥも出来る。

アンドゥ後に Shift+Ctrl+Zで リドゥ出来る。

実行結果

日本工学院専門学校
デザインカレッジ



アドオンを有効化することで、

MyMenu（自分で追加したサブメニュー）に、
「頂点を伸ばす」という項目が出現する。
(テキストは `bl_label`由来)

登録するクラスのリスト

日本工学院専門学校
デザインカレッジ

level_editor.py

```
# Blenderに登録するクラスリスト
classes = (
    MYADDON_OT_stretch_vertex,
    TOPBAR_MT_my_menu,
)
```

Blenderに登録するクラスリストに、
今回作成したオペレータを追加登録する。

これによって、Blender上でクラスとクラスIDが紐付けられるので、
クラスID指定でメニューに登録できるようになる。

サブメニュークラス

日本工学院専門学校
デザインカレッジ

level_editor.py

```
# トップバーの拡張メニュー
class TOPBAR_MT_my_menu(bpy.types.Menu):
    # Blenderがクラスを識別する為の固有の文字列
    bl_idname = "myaddon.topbar_mt_my_menu"
    # メニューのラベルとして表示される文字列
    bl_label = "MyMenu"
    # 著者表示用の文字列
    bl_description = "拡張メニュー by " + bl_info["author"]

    # サブメニューの描画
    def draw(self, context):

        # トップバーの「エディターメニュー」に項目（オペレータ）を追加
        self.layout.operator(MYADDON_OT_stretch_vertex.bl_idname,
                             text=MYADDON_OT_stretch_vertex.bl_label)

    # 既存のメニューにサブメニューを追加
    def submenu(self, context):

        # ID指定でサブメニューを追加
        self.layout.menu(TOPBAR_MT_my_menu.bl_idname)
```

前回追加した
サブメニューに、
自作オペレータを
登録する。

self.layout.operator()
を呼び出すごとに、
メニューに
オペレータ（項目）が
増える。

オペレータクラス

日本工学院専門学校
デザインカレッジ

```
level_editor.py

#オペレータ 頂点を伸ばす
class MYADDON_OT_stretch_vertex(bpy.types.Operator):
    bl_idname = "myaddon.myaddon_ot_stretch_vertex"
    bl_label = "頂点を伸ばす"
    bl_description = "頂点座標を引っ張って伸ばします"
    #リドゥ、アンドゥ可能オプション
    bl_options = {'REGISTER', 'UNDO'}

    #メニューを実行したときに呼ばれるコールバック関数
    def execute(self, context):
        bpy.data.objects["Cube"].data.vertices[0].co.x += 1.0
        print("頂点を伸ばしました。")

    #オペレータの命令終了を通知
    return {'FINISHED'}
```

`bpy.types.Operator` を継承して新たなオペレータのクラスを定義する。

`execute()` がこのクラスの本体。

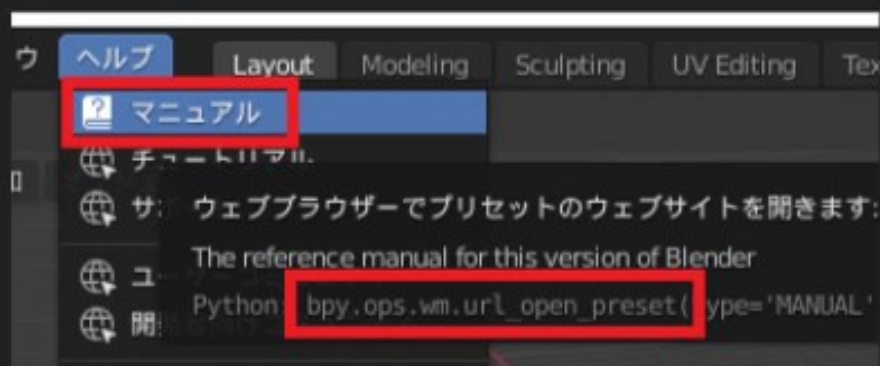
オペレータとして実行させたい処理を記述する。

以前 にスクリプトで行った処理を、オペレータで実装してみる

Operator自作

既存のオペレータ

日本工学院専門学校
デザインカレッジ



BlenderのUIにあるほとんどの機能は、Pythonによるアドオンとして提供されている。

例えば、左図のように「ヘルプ」→「マニュアル」にカーソルを合わせると、ポップアップの説明内に

「`bpy.ops.wm.url_open_preset()`」

とオペレータクラスの名前が表示される。

※バージョンアップによって表示されなくなった。